

 Institución Universitaria Acreditada en Alta Calidad	GUÍA DE TRABAJO PROGRAMA: DESARROLLO DE SOFTWARE	Código	
		Versión	01
		Fecha	2020-01-29

GUÍA DE TRABAJO # 1- LÓGICA DE PROGRAMACIÓN Y LABORATORIO - parte teórica
Conceptos generales, Conceptos de POO I (clase) y Estructura algorítmica I –
Algoritmos Secuenciales

1. IDENTIFICACIÓN DE LA ASIGNATURA

Asignatura:	Lógica de Programación y Laboratorio						
Código:	000506001			Área:	Básicas de la Tecnología		
Plan de Estudios:	1			Semestre:	2		
Créditos: 5	TPS: 6	TIS: 9		TPT: 96	TIT: 144		
Trabajo Independiente:				Trabajo Presencial:			
Trabajo Teórico	X	Trabajo Práctico	X	Trabajo Teórico	X	Trabajo Práctico	X

2. TEMA Y COMPETENCIA

Contenido Temático:	Conceptos generales y Estructura algorítmica I – Algoritmos Secuenciales
Competencia a Desarrollar:	C4- Capacidad para Comunicar de manera clara y efectiva en diferentes contextos profesionales, incluyendo equipos multidisciplinarios, interculturales e internacionales. C6- Capacidad para trabajar de manera eficiente como miembro o líder en equipos multidisciplinarios, aplicando buenas prácticas de colaboración y estándares tecnológicos en la gestión de proyectos de software. C7- Capacidad para Integrar nuevos conocimientos en su práctica profesional, utilizando estrategias de aprendizaje efectivas según las demandas del entorno.
Resultado de aprendizaje:	✓ Participa en espacios colaborativos de análisis y diseño de algoritmos, comunicando sus ideas de manera clara y argumentada, y retroalimentando las propuestas de sus compañeros en entornos de aprendizaje activo. ✓ Asume compromisos en el desarrollo de actividades prácticas y proyectos de lógica de programación, gestionando su tiempo y responsabilidades individuales dentro del trabajo cooperativo con enfoque en metas comunes. ✓ Emplea recursos digitales y estrategias autónomas de aprendizaje para profundizar en la comprensión y aplicación de estructuras algorítmicas, demostrando disposición para aprender de manera continua y adaptativa.

3. RECURSOS REQUERIDOS

Para llevar a cabo las actividades propuestas en esta guía, se requiere de aula de clase cómoda, aireada con tablero; equipo de cómputo con software editor de diapositivas (PowerPoint), Excel y Bloc de notas. Proyector de video con conexión al computador. Material bibliográfico y marcadores varios.

4. MARCO TEÓRICO

4.1 Objetivo

En este curso aprenderemos el propósito básico para iniciar el pensamiento lógico computacional y las estructuras básicas para el desarrollo de algoritmos (Pensar y Entender). Se realizará un análisis y una propuesta de escritura para la solución a un problema planteado; y que luego pueda ser validado y traducido a cualquier lenguaje de programación acorde a su sintaxis. La AI te puede resolver muchos problemas de manera fácil sin pensar en la solución, la desventaja es intervenirlo, sesgo algorítmico y limitaciones creativas entre otros.

4.2 Metodología

En clase, la escritura a emplear es **genérica** (pseudocódigo) ya que es la base para la traducción a cualquier lenguaje de programación.

En laboratorio, se valida lo explicado en clase (algoritmo), transformándolo a un lenguaje de programación de alto nivel (Java); utilizando la sintaxis de dicho lenguaje bajo el paradigma de la POO, utilizando la IDE: Visual Studio Code.

4.3 Problema y Algoritmo

✓ ¿Qué es un Problema?

Es una cuestión, inquietud o necesidad que hay que solucionar (resolver).

✓ ¿Qué es un Algoritmo?

Es el conjunto de instrucciones o serie de pasos que describen el proceso a seguir, en un orden lógico, para encontrar y dar solución a uno o varios problemas específicos y/o para dar uno o varios resultados en especial.

La escritura se realiza de manera textual e independiente de cualquier lenguaje de programación para solucionar un problema.

Nota: Para un problema, pueden existir diferentes formas de expresar una o varias soluciones o respuestas y, por lo tanto, diferentes algoritmos.

4.4 Estructura de un algoritmo E-P-S

La estructura de un algoritmo EPS (Entrada-Proceso-Salida) se compone de tres fases secuenciales y modulares: **Entrada** (datos iniciales), **Proceso** (operaciones y lógica aplicada) y **Salida** (resultados finales). Es una representación funcional, que define qué datos se necesitan, cómo se transforman y qué se obtiene, facilitando la visualización del flujo lógico.

Componentes básicos:

- Entrada (E): Se identifican los datos esenciales y necesarios para resolver el problema. Son las materias primas del algoritmo (ej. valores, variables).
- Proceso (P): Describe detalladamente cada actividad, paso a paso, operación o fórmula necesaria para transformar los datos de entrada en una solución.
- Salida (S): Muestra los resultados obtenidos tras el procesamiento. Es la información final o la solución al problema planteado.



4.5 Conceptos básicos

Son aquellos conceptos que se deben de tener en cuenta para fundamentar el concepto algoritmo y programación, entre ellos tenemos:

✓ Identificador:

Es el nombre que se le asigna a cada elemento o componente de un algoritmo o programa.

✓ Operador:

Es un símbolo o conjunto de símbolos que representa la aplicación de una función sobre unos operandos.

Generalmente están directamente relacionados con los tipos de datos a operar y el tipo del valor que retornan (del mismo o diferente).

Clasificación En pseudocódigo (este curso)

Aritméticos: +, -, *, /, (** ó ^ (alt+94)), Mod

Lógicos: V (o, disy), ∧ (y, conj), ~ (negación)(alt+126)

Relacionales: >, <, >=, <=, ≠, ==

Asignación: ←, =

Entrada y Salida: **Salida:** Muestre, Imprima o Escriba

Entrada: Lea

Concatenar: , (coma)

Debemos diferenciar una suma aritmética y una concatenación, que, a pesar de usar el mismo signo, realizan distintos comportamientos.

✓ **Expresión:**

Es un fragmento o del código que produce o calcula un nuevo dato.

Generalmente está compuesta con operadores y operandos del programa que puede ser evaluada para producir un valor.

✓ **Palabras Reservadas:**

Son identificadores reservados y predefinidos, que tienen un significado o comportamiento especial y no se deben crear o utilizar en los algoritmos y programas.

En pseudocódigo tenemos:

Cadena, real, lógico, entero, vacío, clase, método, principal, si, entonces, fin, para, hacer, mientras, caso, const, público, privado, bit, byte, verdadero, falso, nuevo, ir, retorne... y otras más.

4.6 Tipos de datos

El tipo de dato es una característica o atributo que indica al programador sobre la clase o naturaleza del dato que se va a manejar. Los tipos de datos en programación clasifican la información que maneja un programa, determinando qué valores pueden tomar las variables o constantes y qué operaciones se pueden realizar con ellas.

Se pueden clasificar en dos grupos:

▪ **Simple**

✓ Numéricos

○ Entero: Números sin decimales (ej. 42-10, 0).

○ Real: Números que incluyen decimales (ej. 3.14-0.01).

✓ Carácter y Cadena (Alfanumérico): Secuencia de caracteres, usualmente entre comillas dobles (ej. "Hola""1234").

✓ Lógico o booleano: Representan valores lógicos: Verdadero o Falso (true/false).

✓ Fecha (d/M/a): Representan valores de tipo tiempo, ya sea fechas, horas o ambas.

✓ Otros más...

▪ **Estructurados**

✓ Arreglos

○ Unidimensionales (Vectores): Colección unidimensional ordenada de elemento, generalmente del mismo tipo.

○ Bidimensionales (Matrices): Colección bidimensional ordenada de elemento, generalmente del mismo tipo.

○ Otras dimensiones: Colección acorde a la cantidad de dimensiones que conforma el arreglo.

✓ Pilas: Estructuras LIFO (Last In, First Out).

✓ Colas: Estructuras FIFO (First In, First Out).

✓ Listas: Secuencia de elementos donde cada uno apunta al siguiente.

✓ Árboles: Son estructuras de datos jerárquicas y no lineales utilizadas para organizar información. Son aplicadas sobre una colección de elementos u objetos llamados nodos.

✓ Archivos: Conjuntos de datos organizados bajo un formato estricto y predefinido, facilitando su lectura, búsqueda y procesamiento por máquinas y humanos

✓ Otros más...

Estos tipos de datos son utilizados en lenguajes de programación fuertemente tipados, ya que algunos lenguajes de programación no los utiliza por su manejo de memoria dinámica.

Para nuestros algoritmos utilizaremos básicamente los siguientes tipos de datos: Entero, Real, Lógico y Cadena para los simples; para los estructurados los Vectores y Matrices, **Y todos los datos tendrán un tipo de dato explícitamente.**

4.7 Variables y Constantes

Son Identificadores que reservan espacios en la memoria acorde al tipo de dato.

▪ **Variable**

Como su nombre indica, puede cambiar de contenido/valor a lo largo de la ejecución de un programa.

▪ **Constante**

Como su nombre indica, no puede cambiar de contenido/valor a lo largo de la ejecución de un programa.

✓ Se debe declarar e inicializar antes de ser usada.

✓ Si se intenta cambiar de valor, inmediatamente muestra un mensaje de error.

✓ De modo genérico, se utiliza una palabra reservada antes de su tipo de dato, pero es la sintaxis del lenguaje de programación quien la determina. En el pseudocódigo es: Const.

✓ Por convención, se nombran en letras mayúsculas.

Nota: No todos los lenguajes de programación requieren el declarar las variables y/o constantes, acorde al valor que se quiere almacenar en la memoria, esto debido al manejo dinámico de la memoria.

➤ Declaración de variables y constantes

▪ Variables

- ✓ Para declarar se utiliza el tipo de dato y el nombre que tendrá en memoria.

Entero *Dia*

Cadena *Nombre, direcCli*

Lógico *Estado, sw*

Real *temp_1, temp_2, _valor*

Errores al declarar:

Entero *cant#Dias, nro#* //Uso de caracteres especiales

Cadena *?Nombre, sub Total* //Uso de caracteres especiales y espacios en blanco

Lógico *A+B* //Uso de operadores

Real *5_2, 5toVal* //Uso en inicio del nombre con números

- ✓ Para declarar e inicializar se utiliza el tipo de dato, el nombre asignando y el valor inicial que tendrá en memoria

Entero *dia* ← 10

Cadena *país* ← "Colombia", *direcc* ← "na"

Lógico *estado* ← *verdadero, sw*

Real *temp_1* ← 35.2, *temp_2* ← 0, *valor* ← 0

▪ Constantes

A diferencia de la variable, esta debe empezar con la palabra reservada: **Const** seguido del tipo de dato, el nombre y asignado el valor que tendrá como constante. (Quemado) Nota: *Generalmente en mayúscula.*

Const Real *PI* ← 3.1415926

Const Real *IVA* ← 19.0

Nota: No todos los lenguajes de programación utilizan constantes.

Nota: No todos los lenguajes de programación requieren establecer explícitamente el tipo de dato (no tipados), por lo tanto, sería, por ejemplo:

dia ← 10

nombre ← "Juan Pablo"

4.8 Tablas de verdad

Las tablas de verdad pueden determinar en qué situaciones un enunciado es falso o bien, cuándo es verdadero. Originalmente surgieron como un método semántico de análisis, este tipo de métodos acentúan el carácter de necesidad de la lógica. Se obtiene un valor lógico como resultado en la evaluación de una expresión de dos operandos de valores lógicos con un operador lógico. A continuación, se observan las tablas de verdad para la disyunción (o lógico), conjunción (y lógico) y la negación.

V (o) Disyunción			Λ (y) Conjunción			~ (Negación)	
p	q	$p \vee q$	p	q	$p \wedge q$	p	$\sim p$
V	V	V	V	V	V	V	F
V	F	V	V	F	F	F	V
F	V	V	F	V	F		
F	F	F	F	F	F		

Ejemplos:

Determinar en cada línea si da falso o verdadero acorde a los siguientes valores base: **A** = 10, **B** = 4, **C** = 2, **D** = 5

- ✓ **B > C**
- ✓ **B <= D**
- ✓ **C < A ∧ C < B**
- ✓ **D < B ∨ C > A**
- ✓ **(A < B ∨ D < C) ∧ C < D**
- ✓ **(A < D ∨ 2 * D < A) ∧ ~(C < A ∨ C < B)**

4.9 Jerarquía o prioridad de los operadores

La jerarquía de operadores determina el orden en el que se resuelven las expresiones cuando se involucran diferentes operaciones. Este orden es el que permite que una expresión cualquiera siempre tenga la misma interpretación ya sea resolviéndola en papel, en calculadora o en el computador.

1. ()
2. ** ó ^
3. *, /, Mod, ~
4. +, -, Λ
5. >, <, >=, ≠, V
6. <=, == (igual que)
7. ← o = (Asignación)

Ejemplos:

Determinar en cada caso el valor de x, tener en cuenta los siguientes valores base: **y** ← 15, **a** ← 30, **m** ← 3, **r** ← 5, **j** ← 9

- ✓ $x \leftarrow 3 \times 3 - 3 \div 3 + 3$
- ✓ $x \leftarrow y + a * m - (r + r)^{m-1} / j + \sqrt{y - 2m}$

4.10 Expresiones algebraicas a algorítmicas

Es el proceso de traducir fórmulas matemáticas tradicionales al lenguaje que una computadora puede entender y procesar. A diferencia de la notación matemática, donde podemos escribir variables juntas para multiplicar o usar niveles para exponentes, las computadoras requieren una secuencia lineal y operadores explícitos. Este proceso es el puente entre el **modelo teórico** (la fórmula) y la **solución práctica** (el software). Un error de paréntesis en esta fase puede causar fallos graves en cálculos financieros, estructurales o científicos.

Para que una computadora entienda una expresión, debemos seguir reglas específicas de escritura:

- **Operadores Explícitos:** En álgebra, $2ab$ significa 2 multiplicado por a y multiplicado por b de una manera implícita. En programación se debe de escribir en forma explícita, por lo tanto, se debe colocar el signo de multiplicación (usualmente $*$) nunca es opcional; por lo tanto, se debe traducir $a: 2 * a * b$ (explícitamente para ser procesado por la unidad lógica aritmética (ULA) del procesador).

La **Unidad Aritmético Lógica (ULA o ALU en inglés)** es el componente central de la CPU responsable de ejecutar operaciones matemáticas (sumas, restas, multiplicaciones) y lógicas (comparaciones, AND, OR, NOT) sobre los datos. Actúa como el "cerebro matemático", procesando información binaria para ejecutar instrucciones de software

- **Linealidad:** Todo debe escribirse en una sola línea. No existen las fracciones con "arriba (dividendo) y abajo (divisor)", ejem: $\frac{3a}{9c}$; se usa el operador $/$, quedando: $3 * a / (9 * c)$.

Ejemplos:

Convertir las siguientes expresiones algebraicas a algorítmicas: Nota: Es convertir, no resolver.

- ✓ $3a - 2b^3$
- ✓ $x^{\frac{a-c}{d+b}} h - a$
- ✓ $\frac{3a-4b}{5c} (\sqrt{a-b})^{5/3}$
- ✓ $\frac{1}{\sqrt[5]{\frac{a-b}{c+2e^2}}} / 3\cos(b)$

4.11 Conceptos de POO I

Programación Orientada a Objetos (POO) u (OOP) Object Oriented Programming según sus siglas en inglés.

Es un paradigma que organiza el código en objetos, que agrupan datos (atributos) y comportamientos (métodos) para modelar entidades del mundo real. El término de POO indica una forma de diseño y una metodología de desarrollo de software, no es un lenguaje de programación.

Se basa en clases (moldes) para crear estos objetos, facilitando la modularidad, reutilización y mantenimiento del software.

➤ Clase

Es una plantilla para la creación de objetos de datos en programación, según un modelo predefinido.

- ✓ La clase es uno de los componentes fundamentales de la Programación Orientada a Objetos.
- ✓ Las clases se utilizan para representar entidades o conceptos de la vida real, bajo un lenguaje con dicho propósito.
- ✓ Cada objeto creado a partir de la clase, se denomina **instancia de clase**.

Nota: En este curso, inicialmente se crearán algoritmos dentro de una clase (tipo ejecutables en consola*) para la solución de un problema o enunciado.

➤ Modificadores de acceso

Son palabras reservadas o símbolos que se usan para especificar la accesibilidad o visibilidad de cualquier componente o miembro de una clase al usuario. (Incluso de la clase misma).

Existen varios modificadores de acceso, entre ellos tenemos:

- ✓ **Público:** Cualquier usuario puede acceder al componente de la clase. Cero, una o muchas veces a cada componente. (En POO, Abstracción) (Permitido).
- ✓ **Privado:** Los componentes de la clase solo pueden ser accedidos o utilizados únicamente por componentes de la misma clase. (No Permitido). En POO, lo escondido(privado) al usuario se llama: **Encapsulación**

4.12 Técnica de Diagramación - Tipo Pseudocódigo

Es la técnica para escribir un algoritmo de forma **descriptiva** en un lenguaje natural, con instrucciones estandarizadas para denotar el significado y el accionar de los pasos que debe seguir un algoritmo para dar respuesta o solución a un comportamiento o problema determinado.

También se puede decir que el pseudocódigo es una técnica de diagramación **narrativa y estructurada** que utiliza lenguaje natural combinado con convenciones de programación para describir algoritmos. Actúa como un puente entre la idea lógica y el código real, permitiendo planificar la estructura de un programa antes de la codificación.

Características clave del Pseudocódigo:

- ✓ **Legibilidad humana:** Está diseñado para ser entendido fácilmente por personas, no por computadoras. Utiliza términos cotidianos (como "Inicio", "Escribir", "Si...entonces") para que sea legible por cualquier persona.
- ✓ **No Ejecutable:** No puede ser procesado directamente por una computadora; es un "código falso" diseñado para la planificación humana.
- ✓ **Independencia del lenguaje:** No sigue reglas sintácticas estrictas de un lenguaje de programación específico (como C++ o Python), lo que permite enfocarse en la lógica.
- ✓ **Estructura:** Utiliza estructuras de control secuenciales, selectivas (si-entonces) e iterativas (bucles). Sigue una jerarquía lógica, a menudo usando sangrías (indentación) para mostrar qué pasos dependen de otros.

Componentes Comunes

- ✓ **Inicio y Fin:** Delimitan el cuerpo del algoritmo.
- ✓ **Declaración de Variables:** Identifica qué datos se usarán.
- ✓ **Entrada de Datos:** Instrucciones para recibir información (ej. "Lea valor").
- ✓ **Procesos:** Operaciones matemáticas o lógicas (ej. "Resultado = A + B").
- ✓ **Estructuras de Control:** Decisiones (**Si/Si No**) y ciclos (**Mientras/Hacer, Hacer/Mientras y Para**).
- ✓ **Salida de Datos:** Muestra el resultado final (ej. "Mostrar resultado").

Ventajas sobre otras técnicas

- ✓ **Ahorro de espacio:** Ocupa mucho menos lugar que un diagrama de flujo complejo.
- ✓ **Fácil de modificar:** Al ser solo texto, se edita rápidamente sin tener que redibujar figuras.
- ✓ **Transición suave:** Facilita enormemente el paso final hacia la codificación real en un lenguaje formal.

Ejemplo de Pseudocódigo (Calcular área de un triángulo acorde a la base y su altura):

```
Inicio
  Real base, altura, area = 0
  Muestre ( "Ingrese la base: " )
  Lea base
  Muestre ( "Ingrese la altura: " )
  Lea altura
  area = ( base * altura ) / 2
  Muestre ( "El área es: ", área )
Fin
```

4.13 Estructuras Algorítmicas

Es el conjunto de pasos lógicos y organizados que permiten manipular datos utilizando variables, constantes, expresiones y funciones especiales para realizar procesos específicos dentro del algoritmo para resolver un problema específico. Básicamente, son las "plantillas" de pensamiento que determinan cómo fluyen las instrucciones dentro de un programa.

Tipos Básicos

- ✓ **Secuenciales:** Cada instrucción se ejecuta línea a línea de forma consecutiva.
- ✓ **Condicionales / Decisión:** Para la toma de decisiones lógicas.
- ✓ **Iterativas / Ciclos:** Para la ejecución de un bloque de instrucciones cero, una o muchas veces.

4.14 Estructuras Secuencial

Una estructura de algoritmo o programa es secuencial, cuando se ejecutan las instrucciones (acciones) una tras otra en modo de secuencia (consecutivas) (Top - Down), es decir que una instrucción no se ejecuta hasta que finaliza la ejecución de la instrucción anterior, y así sucesivamente hasta el fin del proceso.

Las instrucciones se describen de tal forma que, el resultado de la instrucción en ejecución puede o no afectar la siguiente.

<pre>b ← 3 a ← 2.5 * b c ← a ** 3</pre>	<pre>c ← a ** 3 Muestre ("Ingrese vr. de D: ")</pre>
Si afecta	No afecta

5. PROCEDIMIENTO

5.1. Problemas para solucionar

1. Crear un algoritmo para que un alumno obtenga su calificación final(**nF**) en la materia de Lógica. Dicha calificación se compone de los siguientes porcentajes: 55% del promedio de sus tres calificaciones parciales (**nP1**, **nP2** y **nP3**). 30% de la calificación del examen final (**nEF**). 15% de la calificación de un trabajo final (**nTF**). Mostrar el nombre y la calificación final. Notas entre 0 y 5 inclusive. ←
2. Tres personas con nombre (**x1**, **x2** y **x3**) deciden invertir su dinero para fundar una empresa. Cada una de ellas invierte una cantidad distinta (**vrx1**, **vrx2** y **vrx3**). Obtener el porcentaje que cada uno invierte con respecto a la cantidad total invertida (**px1**, **px2** y **px3**). Crear un algoritmo para mostrar para cada uno el nombre, el valor invertido y el porcentaje

de participación para fundar dicha empresa.

3. Crear un algoritmo para “**La Tienda**” que lea de un empleado el nombre (**name**), el código (**cod**), el valor de la hora normal (**vrHora**), la cantidad de horas normales(**kHN**) y de horas extras (**kHE**) trabajadas, el valor de las ventas realizadas (**vrVtas**) y el porcentaje de comisiones (**porComi**). Mostrar el código, el nombre, el valor por horas extras (**vrExtras**), el salario bruto(**sB**) (pago por lo laborado), el valor de la retención por parafiscales (**vrPara**) del 9% (ICBF, SENA, caja de compensación), la retención por seguridad social (Salud y Pensión) (**vrSegSoc**) del 8% (Ambas retenciones sobre el salario bruto) y el salario Neto (**sN**) (salario bruto + comisiones – retenciones). Tener en cuenta que cada hora extra tiene un recargo (**rec**) % sobre la hora normal y las comisiones por ventas (**vrComiVtas**) se aplican al valor de las ventas realizadas. ←

Nota: Suponer valores mayores o iguales a cero en la lectura de entrada acorde a lo deseado.

5.2. Taller

Parte 1: Conversión de expresiones algebraicas en algorítmicas

$\frac{A+B}{X+Y} C/D \sqrt[3]{\sqrt[3]{BA^2}}$	$a+b \frac{b+1}{(c-d)\sqrt{a^2+b^2}} - 1 + \frac{f}{g^{-2}}$
$\frac{1}{b+2} + (b-5)^{5x} - c \left(f - \frac{a}{z} \right)$	$5x^3 \left(3 \left(\frac{5}{3} x - 7y^4 \right) - \frac{\sqrt{3}}{2^3} \right)$
$a - \frac{3b/2c}{a+b^{3a} - \frac{3ab}{6c}}$ $z = \frac{\frac{3b}{2c^{2b} + a^{1/5}}}{ab - \frac{3b}{2c^{2b} + a^{1/5}}}$	$x = \sqrt[1/3a]{\frac{\frac{a}{b} - c^3}{a + \frac{c}{b}} * \frac{3ab - 4/3fh}{c - \frac{d+5e}{f + \frac{g}{2h}}}}$

Parte 2: Realizar los siguientes ejercicios propuestos en Pseudocódigo

1. Leído un número entero, elaborar un algoritmo que imprima su raíz cuadrada y el cuadrado de dicho número.
2. Entrar por pantalla los siguientes datos de un estudiante: código, nombre, Nota1, Nota2 y Nota3. Elaborar un algoritmo que muestre el código, nombre y promedio de las calificaciones ingresadas.
3. Dado el radio(**r**) de un círculo, crear un algoritmo que calcule y muestre su área(**A**) y perímetro(**P**). $P = 2 \pi r$, $A = \pi r^2$
4. Una tienda ofrece un descuento del 15% sobre el total de la compra. Mostrar el valor original, el descuento y a pagar por el cliente.
5. Cuatro personas deciden invertir su dinero para fundar una empresa y cada una de ellas invierte una cantidad de dinero. Hacer un algoritmo que muestre el nombre de cada persona, el valor invertido y el porcentaje de participación en la empresa.
6. En un hospital existen tres áreas: Ginecología, Pediatría y Traumatología. El presupuesto anual del hospital se reparte conforme a la siguiente información: Ginecología 40%, Traumatología 25% y Pediatría 35%. Hacer un algoritmo que muestre la cantidad de dinero que recibirá cada área, para cualquier monto presupuestal.
7. Leer dos números enteros, almacenarlos en dos variables e intercambiar el contenido de ambas variables sin utilizar otra variable auxiliar.
8. Hoy puedo comprar **x** panes de \$500 c/u. Si mañana suben a \$550 ¿Cuántos podré comprar con el mismo dinero y cuánto me sobraría?
9. Calcule y muestre el valor de los intereses que un banco debe entregar a un cliente si este se calcula así:
interés = Capital * (1 + Tasa de interés) – Capital.
10. Convierta una coordenada polar (radio, ángulo) en coordenada cartesiana (X, Y). $X = \text{radio} * \cos(\text{ángulo})$, $Y = \text{radio} * \sin(\text{ángulo})$
11. Lea tres los números A, B, C enteros positivos de una cifra. Elabore un algoritmo que matemáticamente genere con estos números otro número de tres cifras. Además, muestre del número generado su cuadrado y su raíz cuadrada. (No concatenar ya que genera es un dato de tipo cadena)
12. Diseñar un algoritmo que calcule el área de un triángulo en función de las longitudes de sus lados, donde la formula seria:
 $A = \sqrt{p(p-a)(p-b)(p-c)}$, donde, $p = (a + b + c)/2$. Suponer que con dichas longitudes se puede construir un triángulo.
13. Una empresa de servicios públicos cobra por metro cúbico de agua **vrM3** centavos de dólar y por kilovatio/hora de consumo de energía cobra **vrKH** centavos de dólar. Haga un algoritmo para determinar el valor a pagar de una vivienda con base en la diferencia de la lectura anterior y la lectura actual tanto de agua como de energía y el valor total a pagar, todo en pesos colombianos.

6. RÚBRICA DE EVALUACIÓN

Resultado de aprendizaje	Indicador de logro	Rango de valoración
--------------------------	--------------------	---------------------

➤ Sabe responder a los conceptos básicos iniciales de la lógica de programación.	➤ Reconoce los diferentes conceptos generales para la programación, elaboración de algoritmos y la programación Orientada a objetos.	0.0 – 1.5
➤ Diseña soluciones algorítmicas utilizando representaciones estructuradas como diagramas de flujo, pseudocódigo y diagramas de clases, integrando principios básicos del paradigma orientado a objetos y normas de nomenclatura.	➤ Convierte una expresión algebraica en una expresión algorítmica.	0.0 – 1.5
	➤ Describe las sentencias asociadas a la estructura secuencial, usadas para la construcción algoritmos usando pseudocódigo.	0.0 – 2.0

7. BIBLIOGRAFÍA

- Aristizábal, D. y Quiceno, S. (2023). Lógica de programación básica orientada a objetos con ejercicios resueltos (1.ª ed.). Editorial ITM. <https://catalogo.itm.edu.co/gpd-logica-de-programacion-basica-orientada-a-objetos-con-ejercicios-resueltos-9789585122710-6491e92907d03.html>
- Efraín M. Oviedo Regino, Lógica de Programación, (2012), Ecoe Ediciones, Bogotá, 332 páginas.
- Joyanes Aguilar, Luis, (2008), Fundamentos de Programación, Mcgraw-Hill/Interamericana De España, S.A, España, 936p.
- Joyanes Aguilar, Rodriguez Baena, Luis, Luis, otra, Fundamentos de Programación Libro de Problemas, Mcgraw-Hill/Interamericana De España, S.A Madrid, 392p.
- Caro Pineda, Silvina, Lógica de Programación y Algoritmos, (2003), Uniboyacá, Tunja, 335 p
- Joyce Farrell, Introducción a la Programación Lógica y Diseño (2013), Cengage Learning, México, 355p.

Webgrafía

- Tutorial de JAVA, <http://www.tutorialesprogramacionya.com/javaya/>
- Programación Básica en Java https://personales.unican.es/corcuerp/java/Slides/Prog_BasicaJava.pdf
- Java Básico para Aprendices, <https://editorialeidec.com/wp-content/uploads/2021/06/Java-Basico-Para-Aprendices-ISBN-978-958-53018-8-7.pdf>

Elaborado Por:	Delio Augusto Aristizábal Martinez
Versión:	V2
Fecha	28/01/2026
Aprobado Por:	Recomendación aprobación comité curricular, marzo 19/2026